



`import RepoSwarm;`

Autonomous, emergent DevOps swarms for any codebase.

```
// The Ultimate Workflow
```

```
"What if your server crashes at 3 AM, and by 3:05 AM, an entire team of senior engineers  
has already diagnosed the issue, mapped the dependencies, written a fix, and is just  
waiting for your 1-click approval?"
```

The_Engineering_Crisis



The 3 AM Crisis

MTTR (Mean Time to Resolution) is broken.

Downtime costs enterprises thousands of dollars per minute while humans scramble to read raw logs.



Context Rot

LLMs hallucinate because they don't understand repository dependencies and hidden architectural decisions.



Rigid Pipelines

Current AI workflows shatter when an unexpected error occurs. Real engineering requires dynamic troubleshooting, not hardcoded steps.

Our_Solution.init()

✈️ **Zero-Shot Setup:** Provide a Github token & API keys.
That's it. No manual orchestration required.

👥 **The Deployment:** The system dynamically provisions
a Band Chat Room and invites domain experts: , , , ,

Watchdog

Planner

Coder

Reviewer

Documentation Agent

🌐 **Generic & Scalable:** Works instantly on *any*
repository without manual tuning. It maps the
environment dynamically.

New Session

Participants

Event type

WD 8

Participants +

@Ajitesh Jamulkar

To make company agents, I need a few things:

- Your company GitHub repo URL
- Your hosted app link (this is the health URL base for the watchdog agent)
- (Optional) A GitHub token, if you want the agents to be able to create PRs (public cloning works without it).

Please provide these details.

01:04:33.100 UTC

Ajitesh Jamulkar

@BAND Orchestrator make company agents for repo

<https://github.com/Ajitesh-jam/band-hack-demo.git>

hosted at <http://localhost:8080>

01:04:52.271 UTC

BAND Orchestrator

@Ajitesh Jamulkar Your company agents (watchdog, documentation_agent, commander, planner, coder, reviewer) have been deployed and added to this room.

For features: Create or enter a chat room with commander, planner, coder, reviewer, and documentation_agent, then ask commander.

For incidents: The watchdog will detect hosted app health failures and open a new room.

You can also ask documentation_agent at any time for codebase, documentation, app status, or recent change context.

01:05:07.156 UTC

Type your message...

USERS

AJ Ajitesh Jamulkar

@ajitesh.jam

AGENTS

BO BAND Orchestrator

@ajitesh.jam/band-orchestrator

A helpful assistant connected via the Band SDK

Public | Remote

C coder

@ajitesh.jam/coder

fixes code from sdk

Public | Remote

C commander

@ajitesh.jam/commander

Plans and distribute work to other agents

Public | Remote

DA documentation agent

@ajitesh.jam/documentation-ag...

scribe agent

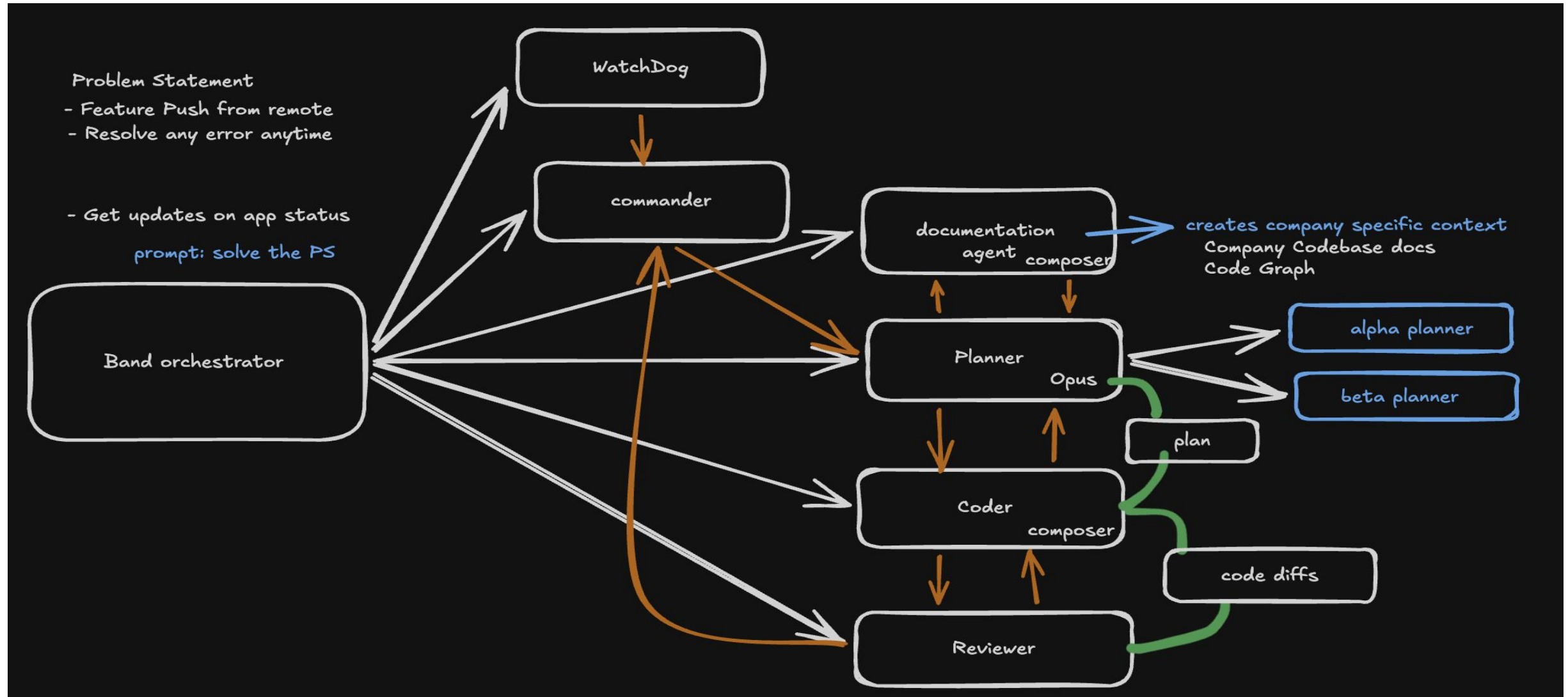
Public | Remote

System_Architecture.yml

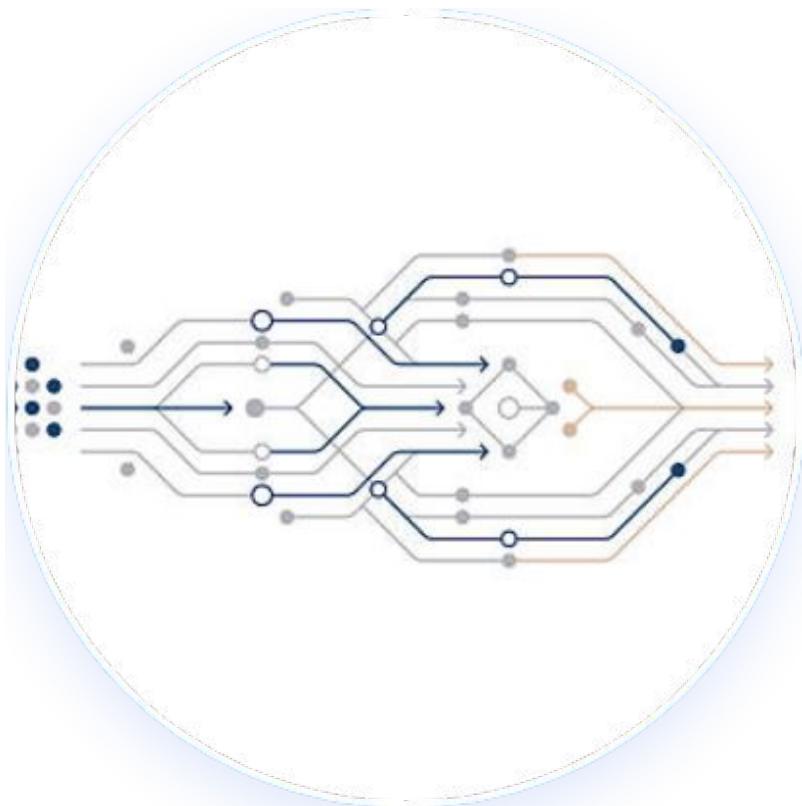
The Flow of Data:

- > **Watchdog** → Continuously monitors and streams active server logs.
- > **Doc Agent** → Provides deep architectural context and dependency graphs.
- > **Planner** → Coordinates the strategy based on logs and docs.
- > **Coder & Reviewer** → Iteratively generate and scrutinize code diffs.

System_Architecture.yml



The_Documentation_Agent



Solving the Generic Context Problem

We completely solved the "Generic Context" problem by making agents environment-aware.

- > It doesn't just do basic RAG. It uses **Graphify** to dynamically map repository dependencies.
- > If a file changes, the agent knows exactly which might break as a result.
- > It learns the codebase organically and acts as the **"Senior Tech Lead"** for the other operational agents.

Action_Demo.mp4



[Video Demo]

```
> Watchdog detects server failure.  
> Watchdog drops log in Band room.  
> Planner consults Doc agent.  
> Coder writes fix, Reviewer approves.  
> Final human 1-click approval for PR.
```

Why_BAND?

❌ Rigid Linear Pipelines

Traditional frameworks require pre-defined workflows and hardcoded orchestrators.

If step 2 fails, the entire script crashes. It cannot handle unexpected real-world engineering environments.

🧩 Emergent Workflows

No rigid orchestrators. The Watchdog spots an anomaly and simply **@mentions** the Planner.

The Planner realizes it lacks context and dynamically **@mentions** the Documentation agent.

If the Coder fails, it doesn't crash a script; it argues with the Reviewer in the chat room until they get it right. **This is impossible outside of Band.**

Experimental_Pursuits.log

Pushing the boundaries of what the platform can do:

- > **Zero-Setup Contacts:** We attempted to make the User Agent a global CONTACT where `users wouldn't even need setup steps at all`
// Note: This extreme friction-reduction requires an Enterprise Plan to implement fully.
- > **Recursive Agent Generation:** We successfully attempted to make a User Agent capable of dynamically spawning multiple, specialized Planner and Coder agents entirely based on reading documentation on the fly.

agent_config.log

Independent agents, each with deep specialization

Every agent in the swarm — Coder, Planner, Documentation Agent, Band Orchestrator — can run completely independently. Each is narrow and specialized on purpose: a generalist agent drowns in context, a specialist agent acts fast and stays accurate.



Planner

Owns strategy only. Reads logs + docs, decides the plan, never touches code directly.



Coder

Owns implementation only. Consumes the dependency map and writes the diff — nothing else.



Documentation Agent

Owns codebase knowledge only. Maintains the live Graphify dependency graph for everyone else to query.



Band Orchestrator

Owns agent creation. Spins up, names, and wires together whichever agents a room actually needs.

The Orchestrator's superpower



On-the-fly agent generation

@BAND Orchestrator can spin up a brand-new, simple agent just from a one-line prompt — no code, no redeploy.



Codebase-aware customization

When deploying company agents, it reads the repo first and tunes each agent's prompts and tools to that specific codebase.



Composable by design

Because every agent is independent, the Orchestrator can mix, match, or replace any one of them without touching the rest.

code_quality.log

Built to be understood, and swapped



Simple, readable structure

We deliberately kept the codebase minimal — thin abstractions, obvious file boundaries, no clever indirection — so any engineer can trace a request end-to-end in minutes.



One harness, any model SDK

Every agent is written against a single common interface. Swap the underlying model SDK — Claude, Gemini, Codex — by changing a config flag, not the agent code.



Config over code

Provider, model name, and tool access are all declared in config. Testing a new model is a flag change and a restart, not a rewrite.

agent.config.yaml

```
agent: coder

# swap providers with one flag — same agent code
model_provider: claude # claude | gemini | codex
model_name: claude-sonnet-4-6

tools:
  - read_repo
  - write_diff
  - run_tests

max_context_tokens: 120000
```

Claude

Gemini

Codex

some agent.py — only the config flag changes



Presentation.log

We will upload every presentation like code explanations, or any more feature implementations in our github repo

For any clarifications regarding code or running you can contact us at

email: ajitesh.jam@gmail.com

discord: cicada3301_1

History.log

```
$ run_monolith_agent.py
> loading full repo context... 480K tokens
! WARNING: context window near limit
> reasoning loop #1...
> reasoning loop #2...
> reasoning loop #3... (repeating)
x ERROR: infinite reasoning cycle detected
$ git log --oneline | head
a1b2c3 fix attempt 14 (broken)
9f8e7d revert revert revert
$ abort. pivoting to swarm architecture_
```



The Monolith Graveyard

Initial iterations attempted a monolithic single-agent script layout. It hit major roadblocks.



The Pitfalls

Context window bloat caused massive LLM hallucinations and infinite reasoning cycles.



The Cost Metric

Spent roughly \$50 in API testing tokens validating and debugging multi-file context boundaries.



The Pivot

Abandoned the monolith mid-hackathon to build a decentralized swarm where micro-agents safely collaborate inside Band.

Business_Value & Future



For Enterprises

Drastically reduces Mean Time to Resolution (MTTR) and prevents context rot. Saves engineering teams sleep and saves companies revenue during outages.



Framework Agnostic

Users can bring their own specialized LangGraph or CrewAI agents into the room to seamlessly swap out our default coders to test performance.

> Band isn't just an API; it's a digital office for AI.



```
rm -rf System32
```

Thankyou :)

Team:

Ajitesh Jamulkar (4 th year Undergrad Student from IIT KGP)

Krishna Paikney (4 th year Undergrad Student from IIT KGP)

Devansh Bavishkar (4 th year Undergrad Student from IIT KGP)